

# Taller — Abandono de producto financiero

Regresión logística en Python · Clase 9

Machine Learning aplicado a Economía y Finanzas · FCEA · 2026-2

24 de septiembre de 2026

## Índice

|                                                        |           |
|--------------------------------------------------------|-----------|
| <b>Presentación</b>                                    | <b>1</b>  |
| <b>Contexto del problema</b>                           | <b>3</b>  |
| Situación . . . . .                                    | 3         |
| Unidad de observación . . . . .                        | 3         |
| Qué se entiende por abandono . . . . .                 | 3         |
| Pregunta de modelación . . . . .                       | 3         |
| Continuidad con la Clase 8 . . . . .                   | 4         |
| Recorrido del análisis . . . . .                       | 4         |
| <b>Base y variables</b>                                | <b>5</b>  |
| Archivo . . . . .                                      | 5         |
| Diccionario . . . . .                                  | 5         |
| <b>Indicaciones de trabajo</b>                         | <b>6</b>  |
| <b>Entregable: repositorio en GitHub</b>               | <b>7</b>  |
| A. Cuenta y repositorio en github.com . . . . .        | 7         |
| B. Git y Visual Studio Code en el computador . . . . . | 7         |
| C. Carpeta local y primer commit . . . . .             | 7         |
| D. Un commit por cada punto . . . . .                  | 8         |
| E. Qué se evalúa en el repositorio . . . . .           | 8         |
| <b>Desarrollo</b>                                      | <b>9</b>  |
| 1. Estructura de los datos . . . . .                   | 9         |
| 2. Descriptivo univariado . . . . .                    | 9         |
| 3. Bivariado con la respuesta . . . . .                | 9         |
| 4. De la recta al logit . . . . .                      | 9         |
| 5. Varios modelos y <i>odds ratios</i> . . . . .       | 10        |
| 6. Predicción en test y matriz de confusión . . . . .  | 10        |
| 7. Perfiles . . . . .                                  | 10        |
| <b>Cierre</b>                                          | <b>11</b> |

## Presentación

Este enunciado tiene **cuatro bloques**, en este orden:

1. **Contexto.** Qué observa el banco, qué es el abandono y qué probabilidad se estima.

2. **Base y reglas.** Diccionario de variables, entorno de Python y criterios de entrega.
3. **Repositorio.** Cómo crear el proyecto en GitHub, trabajarlo en Visual Studio Code y versionar cada punto.
4. **Desarrollo.** Siete secciones de análisis (preguntas 1 a 21), alineadas con la Clase 8.

Las respuestas se escriben **dentro del cuaderno**, en celdas Markdown debajo de cada bloque de código. No invente cifras: use las que produzca **su** corrida.

**Entregable.** La **URL** de un repositorio de GitHub con el notebook **ejecutado** y el CSV. No envíe el `.ipynb` suelto.

## Contexto del problema

### Situación

Una entidad de **banca de personas** ha registrado, en un corte reciente, un aumento de clientes que **cierran la relación** (cuenta o producto). El área de retención necesita pasar de la tasa agregada de la cartera a una **probabilidad individual**: qué cliente, con qué perfil, tiene mayor  $P(\text{abandono} = 1 \mid x)$ .

Ese cálculo no sustituye la decisión comercial. Sirve para **ordenar** la cartera: a quién contactar primero, con qué umbral se declara «en riesgo» y qué error se acepta (llamar a quien se iba a quedar frente a no ver a quien sí se va).

### Unidad de observación

La base `abandono_producto_financiero.csv` es un **corte transversal**. Cada fila es **un cliente** observado una vez, con:

- atributos demográficos (edad, sexo, país de residencia);
- atributos de la relación con el banco (antigüedad, número de productos, tarjeta, marca de miembro activo);
- atributos de posición financiera (puntaje crediticio, saldo, salario estimado);
- el **desenlace ya ocurrido**: permaneció o abandonó.

No es un panel: no hay fecha de cierre ni trayectoria mes a mes. El modelo se estima sobre clientes cuyo  $y$  ya se conoce y, después, se usa para **asignar probabilidad** a perfiles (u otros clientes) que aún permanecen.

### Qué se entiende por abandono

En este taller, **abandono** (*churn*) es el cierre de la relación con el banco, no el atraso de una cuota ni un *default*. La variable respuesta es binaria:

| Código                    | Significado                                   |
|---------------------------|-----------------------------------------------|
| <code>abandono = 1</code> | El cliente <b>se fue</b> (cerró la relación). |
| <code>abandono = 0</code> | El cliente <b>permanece</b> .                 |

La clase 1 suele ser **menos frecuente** que la 0. Eso importa al leer la exactitud: un clasificador que siempre diga «permanece» puede acertar en la mayoría de los casos y, aun así, no detectar a casi nadie que se va. El taller pide verlo en la sección 2 y otra vez en la matriz de confusión (sección 6).

### Pregunta de modelación

Es la misma familia que en la **Clase 8** (crédito alemán), con otro desenlace:

Dadas la edad, el país, el sexo, el saldo, los productos y el resto de  $x$  del diccionario, ¿cuál es  $P(\text{abandono} = 1 \mid x)$ ?

En la Clase 8 el  $y$  era `good / bad` y se estimaba con `smf.logit`. Aquí el  $y$  es **abandono**. El modelo lineal de probabilidad (`smf.ols`) se usa solo como contraste: la recta puede salir del intervalo  $[0, 1]$ . El logit fuerza

$$P(\text{abandono} = 1 \mid x) = \Lambda(x\beta) = \frac{e^{x\beta}}{1 + e^{x\beta}} \in (0, 1).$$

Los coeficientes se leen en **log-odds**. El *odds ratio*  $e^{\beta_j}$  compara el *odds* de abandonar cuando  $x_j$  sube en una unidad (o cuando se activa una dummy), *ceteris paribus*. Un **OR mayor que 1** se lee como factor de **riesgo** (más abandono); un **OR menor que 1**, como factor **protector**.

## Continuidad con la Clase 8

El cierre es el mismo gesto que al final de esa clase: **partir**, **predecir** en test, cortar con un umbral, armar la **matriz de confusión** y calcular exactitud, sensibilidad y especificidad.

|            | Clase 8                                    | Este taller                                 |
|------------|--------------------------------------------|---------------------------------------------|
| Caso       | Otorgamiento de crédito<br>(German Credit) | Abandono de producto / relación<br>bancaria |
| Respuesta  | good / bad                                 | abandono (0 / 1)                            |
| Estimación | smf.logit (y smf.ols como<br>contraste)    | Las mismas funciones                        |
| Factores   | C(·) en la fórmula                         | C(pais), C(sexo)                            |
| Partición  | 70 / 30 con default_rng(8)                 | El mismo sorteo                             |
| Cierre     | Matriz de confusión e indicadores          | Igual, umbrales 0.5 y 0.4                   |

Puede apoyarse en `Regresion_Logistica_German_Credit_Python.ipynb`: `head`, `describe`, histogramas, `boxplot`, `smf.ols`, `smf.logit`, `predict`, `crosstab`.

## Recorrido del análisis

| Sección | Qué se hace                                                   | Preguntas |
|---------|---------------------------------------------------------------|-----------|
| 1       | Cargar, tipificar y contar variables                          | 1–4       |
| 2       | Univariado de $y$ y de las $x$                                | 5         |
| 3       | Bivariado con <b>abandono</b> (boxplots, tablas, ji-cuadrado) | 6–10      |
| 4       | De la recta OLS al logit simple ( <b>edad</b> )               | 11–13     |
| 5       | Varios logits, AIC, <i>odds ratios</i>                        | 14–16     |
| 6       | Test, umbral, matriz de confusión e indicadores               | 17–18     |
| 7       | Dos perfiles y lectura de riesgo                              | 19–21     |

Al terminar cada sección: `commit` y `git push` (mensajes Punto 1 ... Punto 7).

## Base y variables

### Archivo

- Nombre: `abandono_producto_financiero.csv` (UTF-8).
- Debe quedar **junto** al cuaderno, en la raíz del repositorio.
- Identificadores que **no** entran a la fórmula: `numero_fila`, `id_cliente`, `apellido`.

### Diccionario

| Variable                        | Qué registra                                                 | Tipo       | Uso en el logit                            |
|---------------------------------|--------------------------------------------------------------|------------|--------------------------------------------|
| <code>puntaje_crediticio</code> | Puntaje crediticio del cliente                               | Numérica   | Continua                                   |
| <code>pais</code>               | Residencia: <b>Francia</b> , <b>Alemania</b> , <b>España</b> | Categórica | <b>C(pais)</b> ; referencia <b>Francia</b> |
| <code>sexo</code>               | Mujer o Hombre                                               | Categórica | <b>C(sexo)</b> ; referencia <b>Mujer</b>   |
| <code>edad</code>               | Edad en años                                                 | Numérica   | Continua                                   |
| <code>antiguedad</code>         | Años de relación con el banco                                | Numérica   | Continua                                   |
| <code>saldo</code>              | Saldo de la cuenta                                           | Numérica   | Continua                                   |
| <code>numero_productos</code>   | Productos financieros que tiene el cliente                   | Numérica   | Continua (entero)                          |
| <code>tiene_tarjeta</code>      | 1 si tiene tarjeta de crédito; 0 si no                       | Binaria    | 0 / 1                                      |
| <code>miembro_activo</code>     | 1 si figura como activo; 0 si no                             | Binaria    | 0 / 1                                      |
| <code>salario_estimado</code>   | Salario anual estimado                                       | Numérica   | Continua                                   |
| <code>abandono</code>           | 1 = abandonó; 0 = permanece                                  | Binaria    | <b>Respuesta y</b>                         |

`pais` y `sexo` se convierten a `category` con **esos** niveles, en ese orden, para que **C(pais)** y **C(sexo)** usen las mismas referencias que este enunciado. `abandono` se deja como entero 0/1.

## Indicaciones de trabajo

1. Runtime de **Python**. Paquetes de clase: `pandas`, `numpy`, `matplotlib`, `statsmodels`, `scipy`.
2. Trabaje desde **Visual Studio Code** (o Cursor), con `Taller_Abandono_Producto_Financiero_Estudiantes.ipynb` o un cuaderno propio.
3. El CSV va **junto** al cuaderno, con el nombre `abandono_producto_financiero.csv`.
4. Factores con niveles fijos: `pais = Francia, Alemania, España`; `sexo = Mujer, Hombre`.
5. Partición 70 / 30 con `numpy.random.default_rng(8)` (el mismo sorteo que en la Clase 8).
6. Justifique con los números que salgan de **su** corrida.
7. El trabajo se versiona en Git y se publica en GitHub. Un *commit* por cada punto del desarrollo (secciones 1 a 7).

## Entregable: repositorio en GitHub

El producto final es el notebook en **su** repositorio. Siga este orden: crear el repo en la web, abrirlo en Visual Studio Code, ir haciendo *commits* al terminar cada punto y, al cierre, subir (**push**) a GitHub.

### A. Cuenta y repositorio en github.com

1. Entre a <https://github.com> e inicie sesión (o cree una cuenta con el correo institucional).
2. Arriba a la derecha: **+** → **New repository**.
3. **Repository name** (sin espacios), por ejemplo: **taller-abandono-producto-20262**.
4. Descripción opcional: **Taller Clase 9 - abandono de producto financiero**.
5. Visibilidad: **Public** o **Private**, según lo pida el profesor. Si es privado, agregue al docente como colaborador (**Settings** → **Collaborators**).
6. **No** marque *Add a README*, *Add .gitignore* ni licencia (el proyecto ya existe en su computador).
7. Pulse **Create repository**. Deje esa pestaña abierta: ahí aparecen la URL y los comandos **git remote**.

Anote la URL, de la forma [https://github.com/SU\\_USUARIO/taller-abandono-producto-20262.git](https://github.com/SU_USUARIO/taller-abandono-producto-20262.git).

### B. Git y Visual Studio Code en el computador

1. Instale Git si aún no lo tiene. En una terminal: **git --version**.
2. Instale Visual Studio Code (o use Cursor). Extensión recomendada: **Python** (Microsoft) y el visor de Jupyter.
3. Una sola vez, identifique sus *commits* (use el correo de GitHub):

```
git config --global user.name "Nombre Apellido"
git config --global user.email "su_correo@javerianacali.edu.co"
```

### C. Carpeta local y primer commit

1. Copie a una carpeta **nueva** (no entregue **Taller\_solucion**) estos archivos del material:
  - **Taller\_Abandono\_Producto\_Financiero\_Estudiantes.ipynb** (puede renombrarlo a **Taller\_Abandono\_Producto\_Financiero.ipynb**)
  - **abandono\_producto\_financiero.csv**
2. En VS Code: **File** → **Open Folder...** y elija esa carpeta.
3. **Terminal** → **New Terminal** (la terminal debe estar *en* esa carpeta). Ejecute:

```
git init
git add Taller_Abandono_Producto_Financiero.ipynb abandono_producto_financiero.csv
git commit -m "Punto 0: proyecto inicial y base de datos"
git branch -M main
git remote add origin https://github.com/SU_USUARIO/taller-abandono-producto-20262.git
git push -u origin main
```

Sustituya **SU\_USUARIO** y el nombre del repo. Si GitHub pide autenticación, use un **Personal Access Token** (**Settings** → **Developer settings** → **Fine-grained o classic token**) en lugar de la contraseña de la cuenta.

Cree un archivo **.gitignore** en la misma carpeta, con al menos:

```
.ipynb_checkpoints/
__pycache__/.
.venv/
.env

git add .gitignore
git commit -m "Punto 0: gitignore"
git push
```

## D. Un commit por cada punto

Al **terminar** cada sección (código corrido y respuestas escritas en Markdown), no espere al final. En la terminal de VS Code:

```
git add Taller_Abandono_Producto_Financiero.ipynb
git commit -m "MENSAJE"
git push
```

| Cuándo                                  | Mensaje sugerido del commit                |
|-----------------------------------------|--------------------------------------------|
| Tras la sección 1 (estructura)          | Punto 1: estructura de los datos           |
| Tras la sección 2 (univariado)          | Punto 2: descriptivo univariado            |
| Tras la sección 3 (bivariado)           | Punto 3: analisis bivariado                |
| Tras la sección 4 (OLS y logit simple)  | Punto 4: de la recta al logit              |
| Tras la sección 5 (varios modelos)      | Punto 5: modelos y odds ratios             |
| Tras la sección 6 (matriz de confusión) | Punto 6: matriz de confusion e indicadores |
| Tras la sección 7 (perfiles)            | Punto 7: perfiles y cierre                 |

También puede usar la vista **Source Control** de VS Code (icono de ramificación): stage del `.ipynb` → mensaje → **Commit** → **Sync Changes** (equivale a push).

En GitHub, pestaña **Commits**, deben verse **al menos** esos siete puntos (más el inicial). Un único commit al final no cumple el enunciado.

## E. Qué se evalúa en el repositorio

- El notebook **con salidas** (no un archivo en blanco).
- El CSV en la raíz del repo, junto al `.ipynb`.
- Historial de *commits* por punto (tabla de arriba).
- La URL del repositorio es lo que se entrega (aula / correo del curso).

# Desarrollo

Las siete secciones siguientes son el análisis. Cada una cierra con un *commit*.

## 1. Estructura de los datos

Cargue la base. Mire `head`, `columns`, `shape`, `dtypes` y `describe`. Elimine `numero_fila`, `id_cliente` y `apellido`. Convierta `pais` y `sexo` a `category` con los niveles del diccionario. Deje `abandono` como 0/1 (entero).

1. ¿Cuántas variables tiene la base cruda? ¿Cuántas quedan para el modelo?
2. ¿Cuántas son cuantitativas?
3. ¿Cuántas son categóricas o binarias (incluyendo `abandono`)?
4. ¿Qué implica esta mezcla para el logit en Python (`C(pais)`, `C(sexo)` frente a dummies hechas a mano)?

*Commit:* Punto 1: estructura de los datos y git push.

## 2. Descriptivo univariado

En el cuaderno: `value_counts` de `abandono` (conteo y proporción), un gráfico de barras, `describe` e `histogramas` de `edad`, `puntaje_credificio`, `saldo` y `salario_estimado`. Tablas de `pais`, `sexo`, `numero_productos` y `miembro_activo`.

5. ¿Qué porcentaje de la base abandonó? ¿Por qué un clasificador que siempre diga «permanece» ya parece bueno si uno solo mira la *accuracy*?

*Commit:* Punto 2: descriptivo univariado y git push.

## 3. Bivariado con la respuesta

*Boxplots* de `edad`, `saldo` y `puntaje_credificio` frente a `abandono`. Medianas por grupo (`groupby`). Tablas cruzadas y `chi2_contingency` para `miembro_activo`, `numero_productos` y `pais` contra `abandono`.

6. Compare la distribución de `edad` entre quien abandona y quien permanece. ¿Qué dicen el *boxplot* y la mediana?
7. Compare `saldo` y `puntaje_credificio` entre los dos grupos.
8. Relación entre `miembro_activo` y `abandono`. ¿Los clientes activos presentan menor abandono?
9. Relación entre `numero_productos` y `abandono`. ¿Más productos se asocia siempre con menos abandono?
10. ¿Qué variable parece tener mayor relación con el abandono? Justifique con *boxplots*, tablas o ji-cuadrado (no con una impresión).

*Commit:* Punto 3: analisis bivariado y git push.

## 4. De la recta al logit

Hay una  $x$  continua (`edad`) y una respuesta 0/1. Primero `smf.ols`, después `smf.logit`, como en la Clase 8.

Grafique la `edad` contra `y` con un poco de *jitter* y superponga la recta OLS. Trace la curva en S con `predict` del logit.

11. ¿Por qué la recta de mínimos cuadrados no puede leerse como una probabilidad?
12. Escriba el intercepto, el coeficiente de `edad` y el *odds ratio* del logit simple. Interprete el OR en una frase de negocio.
13. ¿Qué  $P(\text{abandono})$  obtiene, con ese logit simple, para un cliente de 40 años?

*Commit:* Punto 4: de la recta al logit y git push.

## 5. Varios modelos y *odds ratios*

Parta 70 / 30 (default\_rng(8)). Estime, de lo simple a lo amplio, y anote el AIC:

- fit1: edad
- fit2: edad + puntaje\_crediticio
- fit3: sume C(sexo)
- fit4: sume C(pais)
- fit5: sume saldo + numero\_productos + miembro\_activo
- fit6: el conjunto del diccionario (todas las variables de modelación)

Quédese con un **modelo de trabajo** (logit\_fits). Pida summary() y np.exp(params).

*Odds ratio* mayor que 1 = factor de **riesgo** (más abandono); **menor que 1** = factor **protector**. Las dummies se leen contra Francia y Mujer.

14. ¿Cuántas filas quedan en *train* y en *test*? ¿Se conserva, más o menos, la tasa de abandono?
15. ¿Con qué especificación se queda y por qué (AIC, parsimonia, *p*)? ¿antigüedad, tiene\_tarjeta o salario\_estimado pagan su puesto?
16. Identifique factores de riesgo y factores protectores del modelo de trabajo. Una frase por variable que importe.

*Commit:* Punto 5: modelos y odds ratios y git push.

## 6. Predicción en test y matriz de confusión

Con el modelo de trabajo:

```
probs = logit_fits.predict(Test_Data)
pred = np.where(probs > 0.5, 1, 0)
pd.crosstab(pred, Test_Data["abandono"], rownames=["Predicho"], colnames=["Real"])
```

Pruebe también el umbral 0.4.

En la tabla, columnas = observado, filas = predicho. Defina:

- **TP**: predijo abandono y el cliente se fue.
- **TN**: predijo que permanece y permaneció.
- **FP**: predijo abandono y el cliente se quedó (contacto innecesario).
- **FN**: predijo que permanece y el cliente se fue (abandono no visto).

Los dos errores no pesan igual para retención: el FN es un cliente que se va sin haber sido marcado; el FP es un contacto que no hacía falta.

$$\text{Exactitud} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Sensibilidad} = \frac{TP}{TP + FN}, \quad \text{Especificidad} = \frac{TN}{TN + FP}.$$

17. A partir de su matriz de test (umbral 0.5), identifique TP, TN, FP y FN. Explique cada uno en el lenguaje del abandono.
18. Calcule exactitud, sensibilidad (recall de abandono) y especificidad. ¿Qué cambia si baja el umbral a 0.4?

*Commit:* Punto 6: matriz de confusion e indicadores y git push.

## 7. Perfiles

Con predict sobre un DataFrame chico, estime  $P(\text{abandono})$  para:

**Perfil A.** Mujer, Francia, edad 40, puntaje 650, antigüedad 5, saldo 100000, 1 producto, sin tarjeta, inactiva, salario 100000.

**Perfil B.** Hombre, España, edad 30, puntaje 720, antigüedad 6, saldo 50000, 2 productos, con tarjeta, activo, salario 80000.

19. Probabilidad del perfil A: \_\_\_\_\_

20. Probabilidad del perfil B: \_\_\_\_\_

21. ¿Cuál es más riesgoso y por qué (qué variables empujan)?

*Commit:* Punto 7: `perfiles y cierre` y `git push`. La URL de GitHub es el entregable.

## Cierre

Este taller pide:

- Situar el abandono como desenlace binario de una relación bancaria y estimar  $P(\text{abandono} = 1 \mid x)$ .
- Cargar la base en español y describir la respuesta y las  $x$ .
- Pasar de la recta (`smf.ols`) al logit (`smf.logit`).
- Elegir un modelo de trabajo con AIC y leer *odds ratios*.
- Cortar las probabilidades en un umbral, armar la **matriz de confusión** y calcular los indicadores, como al final de la Clase 8.
- Publicar el notebook ejecutado en un repositorio de GitHub, con un *commit* por punto.